System Architecture & Technical Documentation

1. Frontend and Backend Tools

Frontend & Backend Framework: Streamlit (Python)
Why Streamlit? Streamlit is a unified framework that serves as both the frontend UI layer and the backend server. It is specifically designed for data-heavy applications.
Specific tasks handled:
- Frontend: Rendering UI components (buttons, sidebars, charts, dataframes) seamlessly without needing HTML/CSS/JavaScript.
- Backend: Handling user inputs, managing state, caching data with @st.cache_data, and orchestrating the data processing pipelines.

Data Processing Tools (Backend logic):
- Pandas & NumPy: Used extensively for data manipulation, ETL processes, and feature engineering. They are optimized for handling large tabular datasets efficiently.
- Pathlib: For robust file system path management.

2. APIs Used and Integration

Currently, the system is fully self-contained and does NOT rely on external third-party REST APIs for fetching core data. It operates entirely on locally generated or uploaded datasets.

However, internally, the system relies on the Plotly API for geographical visualizations (using built-in GeoJSON and Mapbox abstractions) to plot the Top 10 Countries and Provincial Heatmaps.

If a real-world integration were to happen, the system is designed to integrate REST APIs (like a National Identity Database API or Banking APIs) using the 'requests' library in the ETL pipeline layer to stream real-time data instead of reading from static CSV files.

3. Data Models and Algorithms for Training

The system employs multiple machine learning models and algorithms depending on the task:

1. XGBoost (Extreme Gradient Boosting):
   - Use: Risk Classification (Categorizing citizens from A to E).
   - Why: XGBoost is an industry standard for tabular data. It handles non-linear relationships, missing data naturally, and provides excellent feature importance metrics (which features contributed most to the tax

evasion risk score).

2. Isolation Forest (Scikit-Learn):
- Use: Unsupervised Anomaly Detection.
- Why: Used to find outliers (hidden wealth, unexpected transactions) without needing labeled training data. It isolates anomalies by randomly partitioning the dataset.

3. TF-IDF & Cosine Similarity / Levenshtein Distance:
- Use: Entity Resolution (Fuzzy Matching).
- Why: To merge fragmented citizen records where names are misspelled (e.g., Mohd vs Muhammad).

4. Network Analysis Algorithms (Centrality):
- Use: Knowledge Graph construction via NetworkX.
- Why: Identifies hidden syndicates by calculating 'Degree Centrality' (how many connections a node has) to find central figures in tax evasion rings.

4. Function Libraries Explained

The codebase relies on the following key libraries:

Frontend / UI:
- streamlit: The core web framework rendering the dashboard.
- plotly.express / plotly.graph_objects: Used to create interactive, D3.js-based charts (scatter maps, pie charts, bar charts). Specific for rich data visualization.
- pyvis: Used to render the interactive HTML-based network graph (Knowledge Graph).

Backend / Data Processing:
- pandas: Provides the DataFrame object, used for joining tables (vehicles, property, tax), aggregating data, and cleaning.
- numpy: Provides support for fast mathematical operations and handling NaN (missing) values.
- faker: Used strictly in the backend generator to synthesize realistic names, addresses, and vehicle license plates for the sandbox environment.

Machine Learning / AI:
- xgboost: Provides the XGBClassifier class to train the predictive risk model.
- sklearn (Scikit-Learn): Provides preprocessing tools like StandardScaler (to normalize wealth values) and models like IsolationForest.
- networkx: A graph theory library used in the backend to construct nodes and edges connecting citizens, businesses, and properties.